Palturo

IMPLEMENTATION & VERIFICATION · 5 NOVEMBER 2026

Dynamic UX, QOL & Social Presence

What changed, how it was audited, and what is ready to ship.

A complete record of the notification, banner, feedback, chat quality-of-life, privacy, presence, typing and read-receipt work — from the first typed event to the final release gate.



54	155	16	7
COMMITS	TESTS PASSING	PHASES VERIFIED	LIVE MIGRATIONS

BRANCH	PREPARED FOR	METHOD	SCOPE
ui-features @ 08de0e8	**PalTuro product team**	**3 independent audits per phase**	**94 files · 9,635 insertions**

The interface is no longer a set of screens. It is one event system.

Track A	Track B	Track C	Track D
NOTIFICATION EXPERIENCE	QUALITY OF LIFE	PRIVACY AND PRESENCE	OPTIONAL POLISH

CONTENTS

Every section, in reading order.

Thirty-seven fixed pages.
No blank spreads, no orphaned sections.

POSITION

01 Executive summary 03

SCOPE

02 Scope and tracks 04

03 Architecture 05

TRACK A — NOTIFICATION EXPERIENCE

04 A0 · Notification foundation 06

05 A1 · In-app dynamic UX 07

06 A2 · A6 · Feedback layer 08

07 A3 · Notification preferences 09

08 A4 · Local notifications 10

09 A5 · A7 · Push boundary 11

TRACK B — QUALITY OF LIFE

10 B0 · QOL primitives 12

11 B1 · Chat QOL 13

12 B2 · Lists and Activity 14

13 B3 · Contextual polish 15

TRACK C — PRIVACY AND PRESENCE

14 C0 · Privacy contracts 16

15 C1 · C2 · Presence and typing 17

16 C3 · Read receipts 18

TRACK D AND EVIDENCE

17 D · Optional polish 19

18 Data contracts 20

19 Behaviour 21

20 Platform and release 22

21 Test coverage 23

22 Commit ledger 24

23 File inventory 26

24 Limitations 28

25 Operations 29

26 Audit ledger 30

27 Visual evidence 31

28 Acceptance 36

STATUS LANGUAGE

VERIFIED shipped, tested and audited. BLOCKED waiting on an external dependency. DEFERRED intentionally not built.

A phase is only marked verified after the full suite passes and three independent reviews agree.

HOW TO READ THE EVIDENCE

- Scope is frozen at **08de0e8**; documentation commits are excluded.
- Every screenshot is a live build capture, not a mockup.
- Audit session IDs are recorded verbatim in section 16.
- Push delivery (A5) is described as blocked, never as shipped.

> The interface is no longer a set of screens. It is one event system with a shared privacy contract.
>
> This document records what was built, what was measured, and what remains outside the release.

RELEASE POSITION

The app now reacts to the moment.

Live events, chat management and presence share one privacy-aware system.

Fifty-four commits replaced polling and silent state changes with a typed notification backbone, contextual banners, honest preference controls, everyday chat tools, and a server-enforced privacy model for online status, typing and read receipts. The work was delivered in four tracks and closed with 155 passing tests.

54	94	155/155	1
COMMITS IN SCOPE	FILES CHANGED	TESTS PASSING	EXTERNAL BLOCKER

ONE EVENT MODEL

Notifications are typed, deduplicated and routable.

A single service now feeds Activity, in-app banners, local system alerts, the launcher badge and cold-start routing. Unknown payloads fail closed instead of leaking technical text.

QUIET BY DEFAULT

Users choose what reaches them, and when.

Master switch, per-category gates, preview privacy, quiet hours, sound and vibration are all observable and persisted. Presence, typing and receipts are separate, server-backed permissions.

EVERYDAY CHAT

The small actions now exist.

Mute windows, manual unread, drafts, unread divider, scroll-to-bottom count, search, copy, retry and clear-read turn the chat list into a usable inbox.

PROGRAMME AT A GLANCE

TRACK	THEME	ITEMS	EVIDENCE AT CLOSE
A	Notification experience	8 · 1 blocked	122 tests · web release · Android APK · 360/390/1440
B	Quality of life	4 verified	145 tests · live mute, draft, divider, filter and nudge probes
C	Privacy and presence	4 verified	154 tests · live ACL, masking, presence and Seen probes
D	Optional polish	2 · 1 deferred	155 tests · once-per-version live probe

What users will notice

Banners that appear in context, a chat list that can be managed, and status controls that actually remove the status.

What stayed the same

The five-tab shell, the lime navigation, the system theme default and the Pals count treatment were not redesigned.

Sixteen phases verified. One credential-dependent gap. No unresolved defect inside the shipped scope.

Verified with analyzer, 155 tests, a release web build, an Android debug APK, seven live migrations, three independent audits per phase, and captures at 360, 390 and 1440 in light and dark.

BRANCH	WEB BUILD	DATABASE	WORKING TREE
ui-features @ 08de0e8	release, current source	7 migrations applied	clean after push

SCOPE

Four tracks, eighteen work items.

Frozen at 08de0e8. Documentation commits are outside the count.

The programme was planned as a notification track, a quality-of-life track, a privacy and presence track, and a small optional polish track. Each item has a commit range, a verification note and a recorded audit.

TRACK	THEME	ITEMS	OUTCOME	STATUS
A	**Notification experience**	A0–A7	6 verified · 1 blocked	A5 BLOCKED
B	**Quality of life**	B0–B3	4 verified	• COMPLETE
C	**Privacy, presence, receipts**	C0–C3	4 verified	• COMPLETE
D	**Optional polish**	D-01–D-02	1 verified · 1 deferred	D-02 DEFERRED

WORK-ITEM LEDGER

ID	WORK ITEM	STATUS	COMMITS
A0	Notification Foundation	• VERIFIED	8e0f5f3, 5aed01e
A1	In-app Dynamic UX	• VERIFIED	45cf7e7, be90c0d, de4046e, c480fa7, 06f10d8
A2	Haptics and Sound	• VERIFIED	b96c571, 4de648a
A3	Notification Preferences	• VERIFIED	75b1ff8, 72fe744, f89f540
A4	Local Notifications	• VERIFIED	dc5c03c, a2c3fc6, ed9c0ed
A5	Push Delivery	BLOCKED	none
A6	Feedback System Audit	• VERIFIED	8d8f9e6, 445c4c9, 5b97786, da7ff16
A7	Release 1 Verification	• VERIFIED	verification only
B0	QOL Primitives	• VERIFIED	cb7b96a, 27c3355, 9df1965
B1	Chat QOL	• VERIFIED	a24048f, 4b18f8a, fdd641b
B2	List and Activity QOL	• VERIFIED	a25dacd, b8419ca, af0f3c4
B3	Contextual Polish	• VERIFIED	2491fc3, 8110b51, 3e11df1, 12afa6e, 26d38a0, 708316e, bb54994
C0	Privacy Contracts	• VERIFIED	a79ed63, 5c1a4fd, 4aaaa02, 4021a43, 3fa4a45
C1	Presence	• VERIFIED	69bec41, c8bb845, 38949d7, b876970, 487b85a, 5829b91, 27f4784
C2	Typing Indicators	• VERIFIED	354ff68, 8da9b57, 5bb26dc
C3	Read Receipts	• VERIFIED	879e593, 3f468e5, 98e3b59
D-01	What's New	• VERIFIED	08de0e8
D-02	Review Prompt	DEFERRED	none

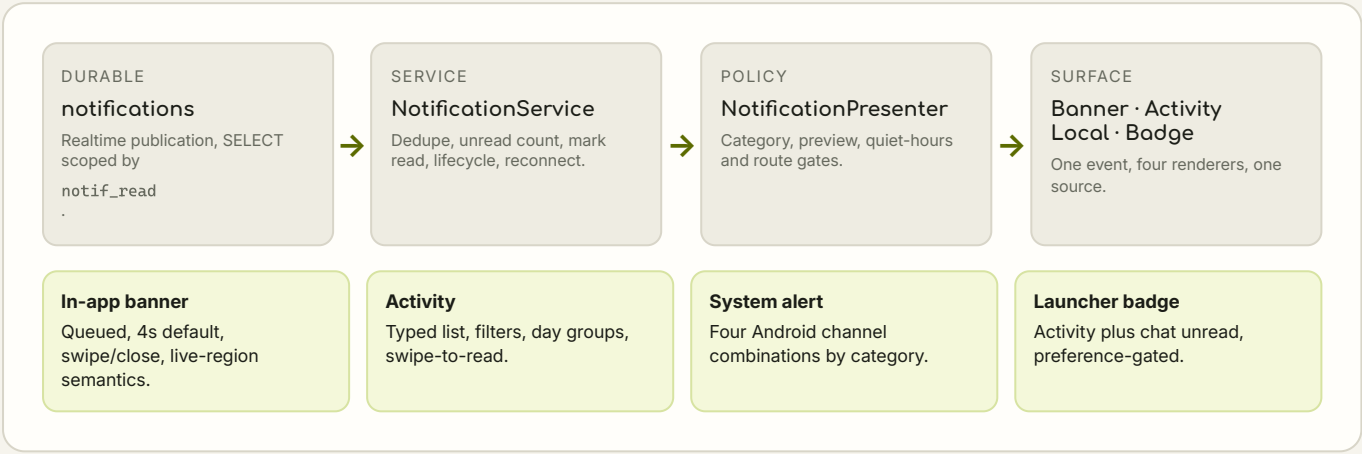
Boundary: A5 push delivery is prepared as a dependency list, not as shipped behaviour. D-02 is deferred because no store listing exists. Neither gap is counted as a completed phase.

ARCHITECTURE

One event path from Postgres to the screen.

Services own state.
Widgets only render it.

The notification and chat systems were deliberately split: durable events arrive through Supabase, ephemeral signals travel over the chat channel, and every surface consumes the same typed state rather than subscribing independently.



EPHEMERAL CHAT CHANNEL

Presence, typing and read signals travel on `chat:<connectionId>`. The client tracks only when the peer allows presence, broadcasts typing with a 2s throttle, and treats read events as hints until the server timestamp is reloaded.

No database writes for typing. No client-clock authority for Seen.

FAILURE POSTURE

Realtime joins refresh an expired session first, then retry with backoff. Privacy state that has not loaded disables the related controls. Unsupported platforms no-op instead of throwing, and sign-out resets every service.

SERVICE INVENTORY

NotificationService
Typed notification state, realtime channel, dedupe, unread list/count, mark read, lifecycle, sign-out reset.

NotificationPresenter
Chooses banner kind, applies category/quiet/preview/privacy gates, plays match/message feedback, suppresses the active screen.

LocalNotificationService
System notifications, Android channels (four sound/vibration combinations per category), permission state/request, tap and cold-start routing.

AppBadgeService
Launcher badge from activity + chat unread, preference gate, serialized writes, web no-op, sign-out reset.

ConnectivityService
Online state, offline strip source, reconnect refresh for notifications and conversations.

DraftService
Per-conversation drafts, debounce, restore, clear on send/sign-out.

ChatPrivacyService
Server-backed presence/typing/receipt switches, coalesced load, optimistic save with revert, reset on sign-out.

Haptics
Named intents, throttle, platform and preference gates, match pattern.

SoundService
Shared player over messages/newmatch/splash assets, preference/lifecycle gates, debounce, serialized init.

AppShellState
Selected tab, Activity/Requests open flags for route-aware suppression.

AppRouter / navigatorKey
Queue-safe routing from services and cold starts.

Rule: no widget talks to Supabase for notification or presence state. Widgets observe a service; the service owns retries, gating and teardown.

TRACK A / A0

A typed event backbone.

One source of truth for Activity, banners, local alerts and badges.

Before A0, unread state was assembled ad hoc and every surface interpreted notification payloads differently. The foundation introduced a typed model, a single realtime service and a router that can act before or after the first frame.

WHAT SHIPPED

- **AppNotification** with known kinds, defensive parsing, payload fallbacks, read state and copy/row adapters.
- **NotificationService** owns the realtime channel, dedupe by id, unread count, mark-one/all, lifecycle refresh, sign-out reset and reconnect.
- **AppRouter** and a global navigator key let banners, Activity, local alerts and cold starts route to Activity, Requests, Chat or Profile.
- **ActivityService replaced** and every caller migrated.
- **Server:** notifications added to the realtime publication with SELECT scoped by the existing read policy; list RPC enriched with actor, skill, role and connection status.

APPNOTIFICATION CONTRACT

KINDS	**request · accepted · declined · feedback · moderation · unknown**
PARSING	**defensive, unknown-safe**
FALLBACKS	**payload → row columns**
STATE	**read / unread, copyWith**
ADAPTERS	**toRow for tests and seeds**

An unknown kind renders as a neutral Activity row. It never throws and never prints raw SQL or payload text.

EVENT ROUTING

KIND	DEFAULT DESTINATION	CONTEXT CARRIED
request	Requests · incoming	`connection_id, actor`
accepted	Chat room	`connection_id, peer, role`
declined	Requests · incoming	`connection_id`
feedback	Activity	`notification_id`
moderation	Activity	`notification_id`

Why it mattered: A1, A3, A4, B2 and C3 all attach to this one model. Without it, each surface would have grown its own parser and its own definition of unread.

REALTIME LIFECYCLE

Join

Refresh an expired session, open the channel, then mark joined only after the subscription acknowledges.

Recover

Back off on JWT or channel errors, resubscribe, and refresh the unread list on success.

Reset

Sign-out clears unread state, cancels the channel and resets the router so the next account starts clean.

MIGRATION CONTRACT

ID	GUARANTEE	FAILURE MODE
001	Realtime insert visible only to the owning user	Row stays in Activity after refresh
002	Activity carries actor, skill, role and status	Neutral row, no technical text

Contract: the payload remains the source of ids; joins only decorate. A missing actor or skill degrades the row, never the unread count.

COMMITS	TESTS AT CLOSE	MIGRATIONS	AUDITS
`8e0f5f3 · 5aed01e`	91	2	3/3

TRACK A / A1

Live events, without leaving the screen.

One banner at a time.
Route-aware, never over the active chat.

A1 made live activity visible in context. The old bottom match sheet was replaced by a top banner system that queues, coalesces and suppresses itself on screens where it would interrupt the task.

STANDARD BANNER

Request, decline, feedback.

Four-second default, close or swipe to dismiss, tap to route, live-region semantics and reduced-motion support. Grouped per conversation so a burst never becomes a stack.

MATCH BANNER

"You're now Pals with X."

Skill line and Say hello action, about seven seconds, once per connection, persisted across restarts. It pre-empts a standard banner and restores the queue afterwards.

MESSAGE BANNER

Sender, preview, tap to chat.

Emitted only for messages outside the active room and not authored by the current user. Same-conversation messages coalesce instead of stacking.

SUPPRESSION MATRIX

CURRENT SURFACE	BEHAVIOUR	WHY
Activity	Notification banners suppressed	The list is already visible.
Requests (pushed)	Request banners suppressed	Avoid duplicate decisions.
Active chat room	That conversation's banners suppressed	Messages are on screen.
Any other surface	Banner shown and queued	Keep the user in context.

REALTIME HARDENING

- ✓ Expired session refreshed before join
- ✓ Backoff on JWT/channel errors
- ✓ Joined state tracked explicitly
- ✓ Recovery on token refresh and resume

ACCESSIBILITY

Banners use a polite live region, expose one primary action, keep 44px targets, and switch to a static presentation when animations are disabled. 360px and 200% text are covered by widget tests.

TIMING AND QUEUE

BANNER	DWELL	QUEUE RULE
Standard	4s	One visible; next waits
Match	~7s	Pre-empts; previous restored
Message	4s	Coalesces per conversation
Nudge	10s	Queued behind the active banner

REDUCED MOTION

When animations are disabled, the banner appears without translation and the match moment uses the static presentation. Grouping still applies, so a burst of messages produces one update rather than a strobe.

Match persistence

A match banner is shown once per connection and remembered across restarts, so reopening the app cannot replay the celebration.

Message coalescing

Several messages from one conversation update the visible banner instead of queuing one per message; different conversations still queue.

COMMITS

`45cf7e7 · be90c0d`
`de4046e · c480fa7 · 06f10d8`

TESTS AT CLOSE

99

BANNER TYPES

3

AUDITS

3/3

TRACK A / A2 · A6

Feedback feels intentional, never technical.

Physical cues and one snackbar contract.
Preferences always win.

Two phases closed the feedback layer. A2 replaced raw haptics with named intents and added gated sound; A6 replaced ad-hoc snackbars and raw error strings with a single feedback helper and a friendly-error allowlist.

A2 · HAPTICS AND SOUND

Named cues, throttled and gated.

- Intents: selection, tap, toggle, impact, success, warning, error, match.
- 50ms per-intent throttle; web/desktop no-op; vibration preference gate.
- SoundService reuses `messages.wav`, `newmatch.wav` and `splash.wav`.
- 150ms debounce, serialized init, stop on route change, lifecycle gate.
- Enabling sound or vibration in Settings previews the choice.

A6 · APPFEEDBACK

One snackbar contract.

- Info, success and error tones with icon and duration (4s / 3s / 6s).
- One action, `clearSnackBars`, and a dialog-scoped variant.
- Adopted across home, chat, matches, requests, block list, explore, account and settings.
- Undo snackbars use an explicit five-second duration.
- Technical leakage rejected before the friendly allowlist.

CUE MATRIX

EVENT	HAPTIC	SOUND	GATE
New match	match pattern	newmatch.wav	category + quiet hours
Incoming message	impact	messages.wav	preview / quiet hours
Request sent or accepted	success	—	vibration preference
Decline or undo	warning	—	single cue only
Destructive confirmation	error	—	vibration preference
Toggle, chip, tab	selection / tap	—	50ms throttle

Leak guard: Postgrest text, SQLSTATE codes, `varying()` errors and auth internals are rejected before the allowlist. Validation copy — including password digit rules and account-deletion messages — is preserved verbatim.

SOUND ASSETS

ASSET	USED FOR
`newmatch.wav`	Match moment only
`messages.wav`	Incoming message while outside the room
`splash.wav`	Startup, once

Single cue

Raw haptics were removed from request cards, so a preference cannot be bypassed and decline does not double-cue.

Preview

Enabling sound or vibration in Settings plays the cue once, confirming the choice without a test screen.

One path: every cue goes through the same preference check. There is no raw platform call left in a widget that could bypass a disabled sound or vibration setting.

A2 COMMITS

`b96c571 · 4de648a`

A6 COMMITS

`8d8f9e6 · 445c4c9`
`5b97786 · da7ff16`

TESTS AT CLOSE

122

AUDITS

6/6

TRACK A / A3

Notification preferences that tell the truth.

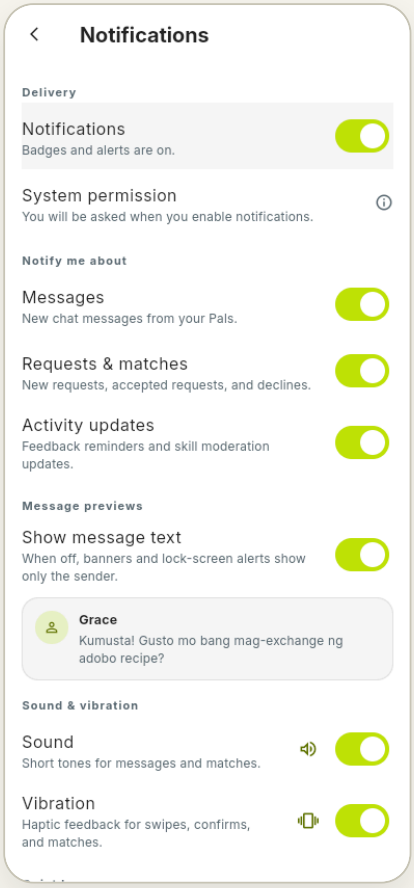
Every switch maps to a real gate.
No fictional OS copy.

A3 turned a thin preference store into an observable model and gave it a dedicated settings page. Each category, preview, quiet-hours window, sound and vibration choice is applied by the presenter before anything reaches a surface.

WHAT SHIPPED

- **Master switch** plus messages, requests/matches, activity, previews, quiet hours, sound and vibration.
- **Presenter gates** each event kind; quiet hours suppress sound and haptics while banners still appear.
- **Preview privacy** replaces message text with “New message” when previews are off.
- **Settings page** with delivery state, permission status, categories, preview example, adaptive time pickers and Pals-only context.
- **Master off** disables categories without erasing their saved values; the Settings root summary stays accurate.

> **Honesty fix:** fictional OS-permission copy was removed. The page reports the actual permission state and never promises delivery it cannot provide.



Notification preferences
Delivery, permission, categories, preview, sound, vibration and quiet hours.

CONTROL	EFFECT	WHEN MASTER OFF
Messages	Message banners and local alerts	Gated off, value retained
Requests / matches	Request, decline and match events	Gated off, value retained
Activity	Feedback and moderation rows	Gated off, value retained
Preview	Message body versus “New message”	Preview hidden
Quiet hours	No sound or haptics; banners remain	No effect

Quiet-hours behaviour

The window is evaluated per event on the device. It suppresses audio and haptics only; it does not mark anything read or hide the banner.

Preview privacy

With previews off, the banner and system alert both read “New message”. The sender remains so the user can decide whether to open the chat.

COMMITS
`75b1ff8` · `72fe744`
`f89f540`

TESTS AT CLOSE
114

WIDTHS
360 · 200%

AUDITS
3/3

TRACK A / A4

System notifications while the process is alive.

Permissions, channels and tap routing.
No claim beyond that.

A4 added local system notifications with real Android channels and a permission model that distinguishes “not asked” from “denied”. Taps and cold-start payloads route into the correct surface with peer context.

PLATFORM CONTRACT

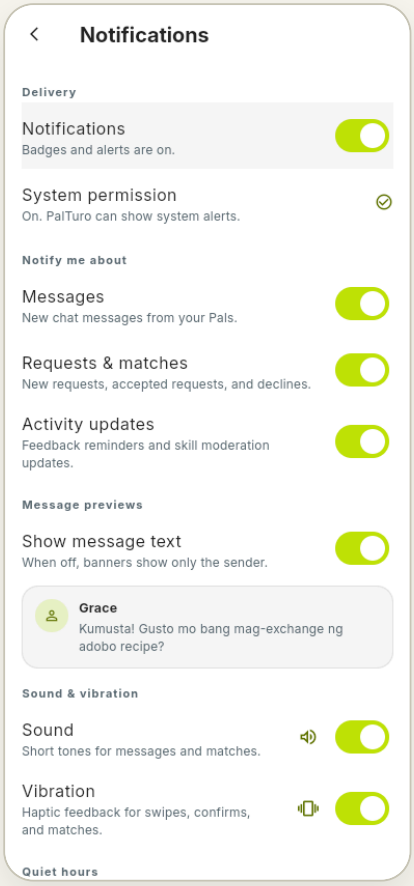
PLATFORM	BEHAVIOUR
Android	Four channel combinations per category (sound × vibration). `POST_NOTIFICATIONS` and `VIBRATE` declared; status icon present.
iOS	`UNUserNotificationCenter` delegate set in AppDelegate; deployment target raised to 13.0. Compiled on macOS still required.
Web	Initialization and permission APIs wired; unsupported browsers fail closed and skip silently.

PERMISSION STATE

Not asked vs denied vs granted. Request from Settings; denied opens system settings; state refreshes on resume.

TAP ROUTING

Chat, Requests, Activity or Profile, including cold start, with the peer and connection context carried in the payload.



Permission state
The page reports the real system state and offers the correct next action.

FOREGROUND

In-app banner is the primary cue; a duplicate system alert is suppressed while the app is visible.

BACKGROUND PROCESS

Local alerts fire while the process is alive. Cancelled on logout and gated by category, quiet hours and previews.

KILLED APP

Not covered. That requires A5 push credentials and is explicitly outside this release claim.

> **Release wording:** notifications work in-app and through a background process on Android. Push for killed apps is prepared but not delivered.

COMMITS
`dc5c03c · a2c3fc6 ed9c0ed`

TESTS AT CLOSE
120

BUILDS
Web + APK

AUDITS
3/3

TRACK A / A5 · A7

Push is prepared, not claimed.

A7 closed release 1 with A5 still blocked.

> **No Firebase project exists. No APNs key exists. Killed-app delivery is not shipped.**
>
> A5 is blocked on external credentials, not on application code. The release must not promise killed-app alerts or guaranteed iOS background delivery.

REQUIRED TO UNBLOCK A5

Firebase project	Android, iOS and web apps registered to the release application id.
`google-services.json`	Android Firebase identity.
`GoogleService-Info.plist`	iOS Firebase identity.
APNs .p8	Key ID and Team ID for Apple push authorization.
Web VAPID	Public key for browser push subscriptions.
Service account	Edge function authentication to FCM HTTP v1.

WHAT STILL WORKS WITHOUT A5

- ✓ In-app banners while the app is open
- ✓ Activity list with realtime inserts
- ✓ Launcher badge from unread counts
- ✓ Local system alerts while the process is alive
- ✓ Server reconciliation on resume and reconnect

The honest product statement is “notifications work in-app and for a background process on Android.”

A7 · RELEASE 1 VERIFICATION

GATE	RESULT	STATE
Analyzer	`flutter analyze --no-pub` — clean	PASS
Tests	`flutter test --no-pub` — 122/122	PASS
Release web	`flutter build web --release` — success	PASS
Android debug	`flutter build apk --debug` — success	PASS
Visual matrix	360/390/1440, dark, home, settings, Activity, banners	PASS
Live probes	Request, match, message and permission states	PASS

A7 outcome: the A track was accepted as release-ready with one documented delivery gap. B and C work continued without pretending the gap was closed.

A5 DELIVERY PLAN

STEP	CHANGE	VERIFICATION
01	Register the release application id in Firebase for Android, iOS and web.	Config files resolve at build
02	Register device tokens and the foreground handler in the client.	Token refresh and logout
03	Send through FCM HTTP v1 from an edge function using the existing event kinds.	Killed-app device test
04	Update the public claim only after Android and iOS device verification.	Release wording review

Do not claim

Killed-app delivery, guaranteed iOS background delivery, or a notification that survives force-stop. None of these is true today.

Reuse, don't rebuild

The event kinds, payload and routing contract already exist. A5 adds transport, not a second notification model.

TRACK B / B0

The app survives a bad connection.

Refresh keeps content.
Offline is visible, not modal.

B0 added the primitives that every later quality-of-life feature depends on: pull-to-refresh that preserves state, a persistent offline strip, a correct launcher badge, and per-conversation drafts.

01
Pull to refresh
Chats, Activity, Requests (incoming and sent) and Browse. On failure the list stays and a snackbar explains the problem.

02
Offline strip
Persistent, laid out above app content rather than over the app bar. Reconnect refreshes notifications and conversations.

03
Launcher badge
Activity unread plus chat unread, master-preference gated, serialized writes, reset on sign-out, skipped on web.

04
Drafts
Per conversation, 350ms debounce, restored on open, cleared on send or sign-out, previewed as “Draft:” in the list.

DRAFT LIFECYCLE

MOMENT	BEHAVIOUR	GUARANTEE
Typing	Debounced write after 350ms of inactivity	No write per keystroke
Leaving chat	Draft persists and appears in the list preview	Survives navigation
Reopening	Composer restores the draft at the cursor end	No lost text
Sending	Draft cleared only after the send is accepted	No empty composer on failure
Sign-out	All drafts cleared	No cross-account leakage

OFFLINE BEHAVIOUR

The strip occupies layout space, so it cannot cover the app bar or intercept taps. Reconnect triggers a notification refresh and a conversation reload instead of waiting for the next manual pull.

BADGE ARITHMETIC

`activity unread + chat unread`, capped by the platform, written serially, and reset on sign-out. Web and unsupported launchers are a no-op rather than a crash.

CONNECTIVITY TRANSITIONS

TRANSITION	BEHAVIOUR
Online → offline	Strip appears above content; current list stays
Offline → online	Notifications and conversations refresh
Refresh failure	Content retained; snackbar explains
Cold start offline	Cached shell renders; actions report

NO SILENT LOSS

A failed refresh never blanks a populated list, and a failed send keeps the draft and the pending row. The user sees the failure and can retry; nothing is quietly discarded.

DRAFT EDGE CASES

CASE	BEHAVIOUR
Send fails	Draft is retained alongside the pending message
Account switch	Drafts are cleared before the next account loads
Conversation deleted	Draft is dropped with the thread

Refresh rule: keep what is on screen, fetch in place, and report failure. A spinner that replaces content is treated as a regression.

COMMITS
`cb7b96a` · `27c3355` `9df1965`

TESTS AT CLOSE
131

SURFACES
4

AUDITS
3/3

TRACK B / B1

Chat does the small things.

Mute, unread, drafts, divider, search, copy, retry.

B1 turned the chat room from a message list into a manageable inbox. Every action is server-persisted where it must survive a reinstall, and purely local where it does not.

MUTE

Quiet one conversation.

1h, 8h, 1d, 1w or always. Muted chats suppress banners and local alerts but still count unread; the list keeps a mute icon.

MARK UNREAD

Return to it later.

Server flag, red dot in the list, cleared when the room opens. It does not fabricate a push notification.

SCROLL FAB

New messages below.

Appears with a count while scrolled up. Auto-scroll only happens when the user is already at the bottom.

COPY

Long-press, one confirmation.

Clipboard write plus a single snackbar. No duplicate system toast.

SEARCH

Find inside the loaded room.

Results with highlight and jump. Scope is the loaded message window, stated in the sheet.

UNREAD DIVIDER

A visible boundary.

First unread message gets a divider, so returning to a long thread has an obvious anchor.

MUTE WINDOWS

DURATION	STORED AS	EXPIRY
1 hour	`muted_until = now + 1h`	Re-evaluated on read
8 hours	`muted_until = now + 8h`	List icon disappears
1 day	`muted_until = now + 1d`	Banners resume
1 week	`muted_until = now + 7d`	Banners resume
Always	`muted_until = null`	Until unmuted

RETRY

Failed pending messages retry sequentially and stop at the first renewed failure, so a dead connection does not spin. Reconnect triggers the same path automatically.

Draft integration keeps the composer and the list preview in sync throughout.

ACTION PERSISTENCE

ACTION	STORED WHERE	SURVIVES REINSTALL	CLEARED WHEN
Mute	`connection_preferences`	Yes	Expiry or unmute
Manual unread	`connection_preferences`	Yes	Room opens
Draft	Local per conversation	No	Send or sign-out
Unread divider	Derived from read state	Yes	Next read

Search scope

Search runs over the loaded message window and says so. It does not imply full-history search that the client cannot guarantee.

Copy behaviour

Long-press writes to the clipboard and raises one snackbar. There is no second system toast and no silent failure.

Persistence: mute and manual unread survive a reinstall because they are server rows. Drafts stay on the device and are cleared on sign-out; they are deliberately not synced across devices.

COMMITS

`a24048f · 4b18f8a fdd641b`

TESTS AT CLOSE

138

MIGRATION

003

AUDITS

3/3

TRACK B / B2

Lists and Activity carry the same rules.

One long-press vocabulary.
One clear-read contract.

B2 pushed chat management and Activity triage to the list level. Actions that were buried in the room became available from the row, and clearing read notifications became a server-side, caller-scoped operation.

CHAT LONG-PRESS

View profile · Mute · Mark unread · Report · Block.

The profile dialog keeps its own block and report paths. Mute and unread reuse the same services as the room, so the list and the thread never disagree.

ACTIVITY FILTERS

All · Unread · Requests · Updates.

Counts on each filter, day groups (Today, Yesterday, date), a filtered empty state, and clear-search recovery. Realtime inserts land in the right group.

CLEAR-READ SEMANTICS

ROW STATE	WHAT CLEAR READ DOES	SERVER GUARANTEE
Read	Deleted for the caller	`read_at is not null`
Unread	Kept, still counted	Never touched
Other user's rows	Invisible	`user_id = auth.uid()`
Failure	List unchanged, error surfaced	No false success

SWIPE TO READ

An unread row can be marked read by swipe. The row updates immediately and reconciles with the server; a failure restores the unread state and reports it.

SAVED FILTERS

Chat role/status and request incoming/sent filters persist per screen, restore on open, and reset through an explicit clear action. A filter can never strand a tab in an empty state with no way back.

Failure honesty: clear, mute and mark-unread report errors instead of pretending success. The previous optimistic-only paths were replaced with a result the surface can render.

ACTIVITY GROUP RULES

GROUP	RULE
Today	Local calendar day
Yesterday	Previous local day
Date	Absolute date, oldest last

Counts

Filter counts come from the same list that is rendered, so a badge cannot disagree with the rows beneath it.

Reset

Clear filters restores the default view and the saved filter is removed, not just visually bypassed.

FILTER PERSISTENCE

SCREEN	SAVED STATE	RESTORED
Chats	Role and status	On open
Requests	Incoming or sent, skill, domain, sort	On open
Any filtered list	Explicit clear removes the saved value	Next open

COMMITS
`a25dacd · b8419ca af0f3c4`

TESTS AT CLOSE
141

MIGRATION
004

AUDITS
3/3

TRACK B / B3

The next step, without the nag.

One prompt at a time.
Every empty state has an action.

B3 handled the moments after a request, after onboarding, and after a filter returns nothing. The rule was consistent: surface one useful next step, never block the committed action, and never stack two completion prompts.

CONTINUE SETUP

Only when setup is incomplete.

Appears when onboarding was skipped or a draft exists. Resume restores the review step; dismiss persists per user and the card returns if setup is skipped again.

PROFILE NUDGE

After the request is committed.

Uses the app banner: non-blocking, SafeArea, queued, closable, ten seconds, at most once per session. It never interrupts the send.

EMPTY STATES

One clear action each.

Start swiping, view sent requests or refresh, view requests or show all, update skills or refresh categories. No dead-end copy.

TRIGGER MATRIX

SURFACE	CONDITION	ACTION
Home	Setup incomplete, skipped or draft exists	Resume review · dismiss per user
After request	Send succeeded, profile incomplete	Profile nudge, once per session
Chats filtered empty	Filter returns no rows	Clear filters or start a chat
Requests sent empty	No outgoing requests	View incoming or refresh
Explore empty	No categories for the role	Update skills or refresh categories

One-prompt rule: Profile shows only one completion prompt at a time. Requests never leaves the tab trapped in the sent view; back always returns to a meaningful screen.

DISMISSAL SCOPE

PROMPT	STORED PER	RETURNS WHEN
Continue setup	User	Setup skipped again
Profile nudge	Session	Next session, once
What's new	Version	New version

Resume path

Resume restores the review step with its saved draft; it does not restart onboarding.

Nudge timing

The request is already committed before the nudge appears, so dismissing it never cancels the send.

Dismissal is not failure

Dismissing the resume card hides the prompt, not the draft. The setup data stays until the user completes or clears it.

One prompt at a time

Profile calculates which completion prompt to show and renders one; two prompts never compete for the same space.

Dead-end rule: every empty state names the next action and the action is reachable without leaving the current tab.

COMMITS

`2491fc3 · 8110b51 · 3e11df1 12afa6e · 26d38a0 · 708316e · bb54994`

TESTS AT CLOSE

145

WIDTHS

360 · 200%

AUDITS

3/3

TRACK C / C0

Privacy is a contract, not a toggle.

Define the data before showing it.
Mask at the server, not the widget.

C0 added the status fields and then closed the paths that would have leaked them. The client cannot read connection read timestamps directly, and every presence or receipt value is masked to accepted Pals with the relevant permission on.

CONTRACTS ADDED

CONTRACT	PURPOSE	ACCESS
`last_seen_at`	Presence timestamp on the profile	self only, via RPC
`presence_enabled`	Peer may see online / last active	self write, masked read
`typing_enabled`	Peer may see typing	self write, masked read
`read_receipts_enabled`	Peer may see Seen	self write, masked read
`touch_last_seen`	Self-only heartbeat	SECURITY DEFINER
`get_pals_presence`	Accepted Pals only; mask when off	authenticated
`set/get_chat_privacy`	Three switches, server-backed	authenticated

CLIENT MAY READ

- ✓ Own privacy settings
- ✓ Peer status for accepted Pals when enabled
- ✓ Peer read timestamp when receipts are on
- ✓ Conversation metadata without receipt columns

CLIENT MAY NOT READ

- × Connection table read timestamps directly
- × Status for non-Pals or declined connections
- × Last seen when presence is off
- × Receipts when the peer has them off

Closing the bypass: migration 007 revokes table SELECT on `connections` and grants only id, participants, status, timestamps and skill columns. Read timestamps are reachable only through RPCs that apply the permission.

MASKING MATRIX

VIEWER	PRESENCE OFF	TYPING OFF	RECEIPTS OFF	NOT A PAL
Accepted Pal	No status	No indicator	No Seen	—
Pending request	No status	No indicator	No Seen	Blocked
Declined	No status	No indicator	No Seen	Blocked
Self	Own value	Own value	Own value	—

Load failure

Switches are disabled until server state loads. A failed load shows Retry and never assumes the permissive default.

Reset

Sign-out resets the privacy service. The next account cannot inherit a previous session's switches.

Why the server owns this

A client-side hide is a suggestion. The masking lives in the RPC and the grant, so a modified client still cannot read a status the peer turned off.

COMMITS

`a79ed63` · `5c1a4fd` · `4aaaa02`
`4021a43` · `3fa4a45`

TESTS AT CLOSE

149

MIGRATIONS

3

AUDITS

3/3

TRACK C / C1 · C2

Online, typing, and nothing more.

Presence is a heartbeat.
Typing is a broadcast, never a write.

Presence and typing were built on the existing chat channel. Both are gated by the peer's privacy flags, both tear down cleanly on leave, and neither exposes a status to someone who is not an accepted Pal.

C1 · PRESENCE

A 60-second foreground heartbeat.

- Heartbeat and conversation refresh while resumed.
- Disabled when privacy has not loaded or presence is off.
- Chat list shows green **Online** or muted **Active Nm/h/d**; message time stays visible.
- Room subscribes to `chat:<connectionId>`, filters the peer by exact user id.
- Channels removed on leave, privacy-off or error without re-entrant teardown.

C2 · TYPING

Three dots, two seconds, zero rows.

- Broadcast on the existing chat channel with a 2s throttle.
- Peer expiry after 3s; stop on empty, send or leave.
- No database writes.
- Gated by the local typing setting and the peer's typing flag.
- Works with presence off: the channel joins without tracking presence.
- Typing appears before the first message; the pre-first-message match card stays.

LIFECYCLE

01 Resume
Session refreshed; privacy state loaded before any track.

02 Heartbeat
Self last-seen advanced every 60s while presence is on.

03 Track
Room joins the channel and tracks only when allowed.

04 Signal
Typing and read broadcasts are throttled and peer-scoped.

05 Teardown
Leave, pause, privacy-off and error all remove the channel.

MASKING RULES

When presence is off, the peer receives no online state and no last-seen value. When typing is off, the indicator never renders. When receipts are off, read timestamps are null even if the rows exist.

EDGE CASES COVERED

Re-enabling presence retracks; typing-only mode still joins; a failed channel recovers; a typing event can arrive before the first message without breaking the match card.

SIGNAL RULES

SIGNAL	TRANSPORT	CADENCE	EXPIRY / TEARDOWN
Presence	Channel track	60s heartbeat	Leave, pause, privacy off
Typing	Broadcast	2s throttle	3s peer expiry
Receipt	Broadcast + reload	Trailing flush	60s refresh
Online label	Server contract	On list refresh	Masked when off

No status leak

A peer who is not an accepted Pal never receives a track, a typing event or a read timestamp, regardless of the local toggle.

No stale state

The 60s heartbeat and the conversation refresh mean a closed app becomes “Active Nm” rather than staying online forever.

COMMITS

`69bec41` · `c8bb845` · `38949d7`
`b876970` · `487b85a` · `5829b91`
`354ff68` · `8da9b57` · `5bb26dc`

TESTS C1 / C2

150 / 152

DATABASE WRITES

0

AUDITS

6/6

TRACK C / C3

Seen means the server says so.

One label, under the last own message.
No device-clock authority.

C3 added read receipts without reopening the bypass C0 closed. A throttled broadcast is only a hint; the displayed time is always reloaded from the server, so a skewed device clock cannot produce a false Seen.

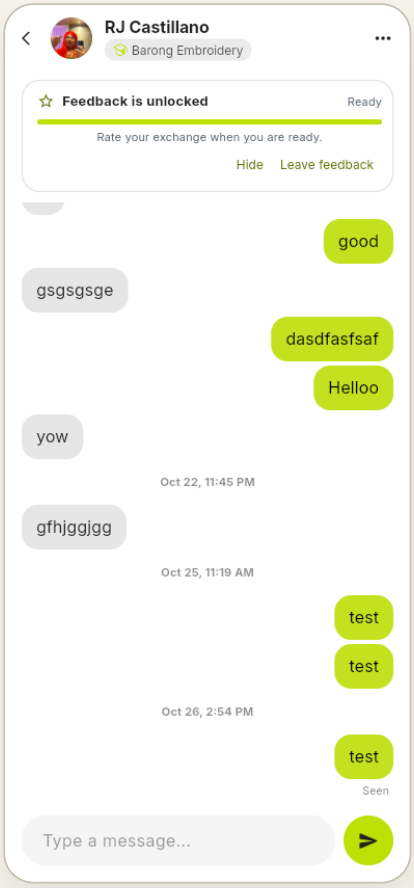
RECEIPT PATH

01
Peer reads
Mark-read commits on the server.

02
Broadcast hint
A throttled read event with a trailing flush reaches the room.

03
Server reload
The client reloads the conversation timestamp; the broadcast value is discarded.

04
Render
Seen appears only under the last own message, when receipts are enabled and the timestamp covers it.



Seen
Rendered under the last own message only, with server-authoritative time.

GATING

CONDITION	RESULT
Peer receipts off	No Seen, even if the read row exists
Older own message	No label; only the last own message qualifies
Peer has not read that far	No label
App backgrounded	Read broadcasts paused; 60s refresh reconciles

Transport note: receipts use a broadcast plus a 60-second conversation refresh instead of publishing `connections`, because C0 made read timestamps RPC-only. A read event can arrive before mark-read commits; the server refresh is the source of truth.

COMMITS
`879e593` · `3f468e5`
`98e3b59`

TESTS AT CLOSE
154

CLOCK SOURCE
Server

AUDITS
3/3

TRACK D / D-01 · D-02

Ship the small wins; defer the fake ones.

One useful sheet. One honest omission.

The final track was intentionally small. D-01 tells returning users what changed once per version. D-02 was deferred because a review prompt without a store listing would lead to a dead link.

D-01 · VERIFIED

What's new, once per version.

A version-gated bottom sheet with three bullets and "Got it". It is scrollable at 360 and 200%, dismissal persists per version, and the sheet does not appear again after reload.

- ✓ Version key stored locally
- ✓ Scrollable at 360 and 200%
- ✓ Dismiss persists per version
- ✓ Live probe: shown → dismissed → absent after reload

D-02 · DEFERRED

A review prompt without a listing is a dead end.

No published store listing exists, so any prompt would be fake or lead nowhere. When a listing exists, the prompt should be gated on a positive feedback or exchange, once per version, and never over a dialog.

No code shipped. This is a product decision, not an unfinished implementation.

RELEASE 2 TOTALS

MEASURE	VALUE	NOTE
Commits	54	3ab6700..08de0e8
Files changed	94	9,635 insertions · 1,301 deletions
Phases verified	16 of 18	A5 blocked · D-02 deferred
Tests	155 / 155	Full suite at D-01 close
Migrations	7	All applied live

The release is complete where it can be complete.

Nothing in the shipped scope is waiting on a missing decision. The two open items are external credentials and a store listing, and both are documented as such.

D-01 SHEET CONTRACT

PROPERTY	BEHAVIOUR
Trigger	First open of a new version
Content	Three bullets, "Got it"
Dismiss	Persists per version
Layout	Scrollable at 360 and 200%

Version gate

The stored key is the version, so a patched release can introduce its own sheet without repeating the last one.

Accessibility

The sheet is scrollable at large text and dismissible from the sheet itself, not only by tapping outside.

Versioned, not global

The dismissal key includes the app version, so a new release can introduce a new sheet without repeating the previous one.

Why D-02 waits

A review prompt needs a destination. Asking before a listing exists creates a dead link and a poor store signal.

COMMIT
`08de0e8`

TESTS
155

VERIFIED
16

DEFERRED
1

DATA CONTRACTS

Seven migrations make the interface true.

All applied live.
All committed.

The UI is only trustworthy because the database owns the hard rules: who may see a status, which read timestamp is exposed, what a mute stores, and whether a clear-read operation can touch another user's rows.

ID	MIGRATION	PURPOSE	CONTRACT
001	0100_notifications_realtime	**Notifications realtime** Adds public.notifications to supabase_realtime (guarded) and grants SELECT to authenticated; row visibility stays scoped by notif_read (user_id = auth.uid()).	notifications → realtime
002	0200_notification_context	**Activity context** Recreates list_notifications with actor name/avatar, skill, role, and connection status joins while keeping payload as the source of ids.	list_notifications
003	0300_connection_preferences	**Chat mute + manual unread** Adds connection_preferences (user_id, connection_id, muted_until, marked_unread), SELECT-only RLS, set_connection_mute, set_connection_unread, get_connection_preferences, and appends muted/marked fields to get_conversations.	connection_preferences
004	0400_clear_read_notifications	**Clear read Activity** Adds clear_read_notifications(), a SECURITY DEFINER delete limited to user_id = auth.uid() and read_at is not null.	clear_read_notifications
005	0500_chat_privacy	**Privacy contracts** Adds last_seen_at and three privacy flags to profiles; adds touch_last_seen, set/get_chat_privacy, get_pals_presence; appends peer presence/typing/receipts/read fields to get_conversations. Realtime topic ACL cannot be installed from the management role (see limitations).	chat_privacy RPCs
006	0600_privacy_status_masking	**Status masking** Recreates get_conversations so presence/typing/receipt flags and last_seen/read timestamps are returned only for accepted connections, and read_at is null when receipts are off.	get_conversations
007	0700_lock_connection_reads	**RPC-only read timestamps** Revokes table SELECT on connections from anon/authenticated and grants only id, requester_id, recipient_id, status, created_at, responded_at, skill_name, skill_path; read timestamps are reachable only through RPCs.	connections grants

7/7
APPLIED LIVE

0
INVALID LIVE PATHS

3
PRIVACY MIGRATIONS

Known boundary: the management role cannot alter `realtime.messages`, which is owned by `supabase_realtime_admin`. Chat topics rely on unguessable connection UUIDs and client-side privacy gating; a dashboard-authored realtime ACL would be required for server-side topic authorization.

READ-ONLY BY DESIGN

Notification reads are scoped by the existing `notif_read` policy. Connection read timestamps are not in the table grant at all; they are reachable only through the RPCs that apply the receipt toggle.

WRITE PATH

Mute, manual unread, privacy switches and clear-read are all RPCs. The client does not construct arbitrary updates against these tables.

BEHAVIOUR

One event, defined for every surface.

In-app, system, badge.
Suppression is explicit.

The same event kind produces a predictable set of outputs. The matrix below is the contract the presenter and local-notification service implement; if a row cannot be satisfied, the event is suppressed rather than partially delivered.

EVENT	IN-APP BANNER	LOCAL ALERT	BADGE	SUPPRESSED WHEN
Request	4s, route to Requests	Category-gated	+1 activity	Requests screen is open
Accepted	Match banner, ~7s	Category-gated	+1 activity	Active chat for that connection
Declined	4s, route to Requests	Category-gated	+1 activity	Requests screen is open
Message	Preview or “New message”	Channel per category	+1 chat	That room is active
Feedback	4s, route to Activity	Activity category	+1 activity	Activity is open
Moderation	4s, route to Activity	Activity category	+1 activity	Activity is open
Unknown	Neutral row only	Never	No change	Always

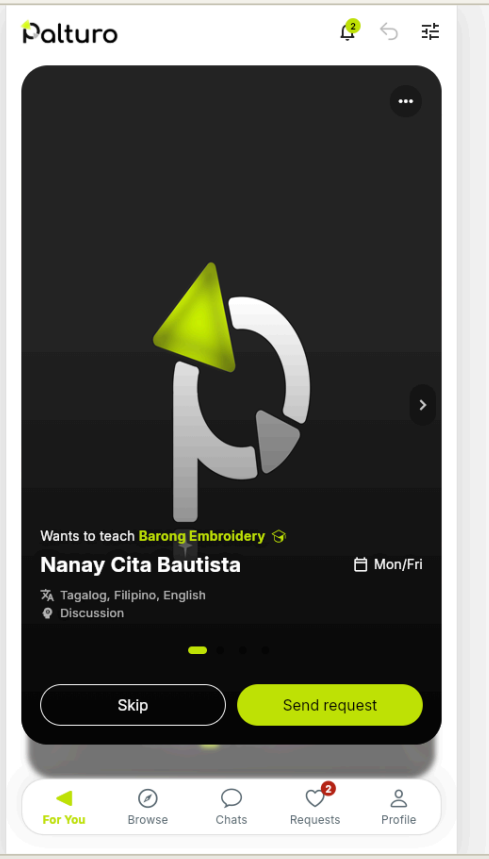
QUIET HOURS

Banners still appear; sound and haptics are suppressed. The window is local to the device and does not change server read state.

PREVIEWS OFF

Message text is replaced with “New message” in both the banner and the system alert. The sender name remains.

DESKTOP SHELL · 1440 × 900



The centred mobile surface is unchanged.
At 1440 the app remains a focused 480px column; the dynamic UX work did not introduce a second layout.

PLATFORM AND RELEASE

Capability is stated per platform.

No shared promise that the code cannot keep.

PLATFORM MATRIX

CAPABILITY	ANDROID	IOS	WEB
In-app banners	FULL	FULL	FULL
Activity + unread	FULL	FULL	FULL
Local notifications	4 CHANNELS	DELEGATE SET	BEST EFFORT
Permission request	RUNTIME	RUNTIME	BROWSER
Launcher badge	SUPPORTED	SUPPORTED	SKIPPED
Presence / typing / receipts	FULL	SOURCE ONLY	FULL
Killed-app push	BLOCKED	BLOCKED	BLOCKED

FINAL GATES

GATE	OBSERVED RESULT	STATE
Analyzer	`flutter analyze --no-pub` — clean	PASS
Tests	`flutter test --no-pub` — 155/155	PASS
Release web	`flutter build web --release` with env defines	PASS
Android debug	`flutter build apk --debug` with env defines	PASS
iOS	Source and config only on this Windows host	MAC REQUIRED
Working tree	Clean after push to `origin/ui-features`	PASS

155/155
TESTS PASSING

0
ANALYZER ISSUES

7
MIGRATIONS APPLIED

Verification matrix: 360 / 390 / 1440 · light and dark · 200% text. Home, Activity, Chats, Chat room, Requests, Browse, Profile, Notifications, Chat privacy and the banner surfaces were all exercised.

BUILD ARTIFACTS

ARTIFACT	SOURCE	RESULT
Release web	`08de0e8 + env defines`	BUILT
Android debug APK	`08de0e8 + env defines`	BUILT
iOS source	`AppDelegate + target 13.0`	NOT COMPILED
Test binary	`155 tests`	PASS

Windows host

Web and Android were built locally. iOS source is verified, but a macOS toolchain is required for a real compile.

Device pass

VoiceOver and TalkBack remain a recommended manual activity; automated semantics and 200% tests are not a substitute.

What the matrix means: a capability marked full has a code path and a test or live probe. A capability marked blocked is intentionally absent and is never described as available.

TEST COVERAGE

155 tests across the new surface.

Unit, widget, semantics and negative paths.

The suite grew from 80 tests at the start of the work to 155 at D-01. Coverage follows risk: event parsing and gating, banner lifecycle, chat state, privacy masking, and the text/width combinations most likely to break.

AREA	FILES	FOCUS
Notification model/realtime	`app_notification_test.dart`	Parsing, unknown kinds, payload fallbacks, read state, presenter adapter.
Banners	`app_banner_test.dart`	Queue, action, dismiss, match preempt, message coalesce, 360/200%, dark surface.
Haptics/sound	`haptics_test.dart`, `sound_service_test.dart`	Intent mapping, preference gate, throttle, real asset presence, volumes.
Preferences	`notification_preferences_test.dart`	Defaults, persistence, quiet-hours windows, category gating, preview privacy, 360/200%.
Local notifications	`local_notification_payload_test.dart`	Deterministic ids, channel matrix, permission state, routing payloads.
Feedback/errors	`app_feedback_test.dart`, `friendly_error_test.dart`	One-at-a-time snackbars, undo, technical leakage, validation preservation.
QOL services	`qol_services_test.dart`, `draft_service_test.dart`, `offline_banner_test.dart`	Badge math, connectivity transitions, draft reload/clear, offline strip semantics/app-bar taps.
Chat QOL	`chat_qol_test.dart`, `chat_widgets_test.dart`	Mute window, manual unread, divider index, search sheet lifecycle, menu 200%, presence/time row, typing payloads/indicator, Seen gating.
Privacy	`chat_privacy_test.dart`	Presence labels/window, page render 200%, load-failure gate, keys.
Polish	`b3_polish_test.dart`, `whats_new_test.dart`	Resume gate/keys, scoped dismissal, card 360/200%, what's new bullets 360/200%.

155	75	10	0
TESTS TOTAL	ADDED IN SCOPE	TEST AREAS	SKIPPED

NEGATIVE PATHS COVERED

- ✓ Raw SQL and Postgrest text rejected
- ✓ Failed clears do not report success
- ✓ Privacy load failure disables controls
- ✓ Malformed notification payload fails closed

KNOWN TEST BOUNDARY

Some interaction paths (name sheet in the earlier release, several request flows, and live presence) are verified by audit and live probes rather than full widget tests. Manual VoiceOver/TalkBack remains a device activity.

Live probes: Notification insert → top banner (light/dark), request accepted → match banner, message insert → message banner. · Mute sheet → muted list icon; mark unread → list dot → open clears it; draft typed → 'Draft:' preview after back. · Unread divider visible before unread rows; scroll-up + peer insert → FAB count; long-press copy → clipboard + snackbar; search → highlight/jump. · Activity filters/counts/day groups; clear-read confirmation; chat long-press action sheet. · Offline strip above the app bar; reconnect restores content; profile nudge banner; one-time what's new. · Presence: peer last_seen set → Online in list and room; own last_seen advanced after app load. · Read receipts: Seen under the last own message.

COMMIT LEDGER I

Commits 1–27 of 54.

3ab6700..08de0e8
Newest last.

#	HASH	DATE	PHASE	SUBJECT
01	8e0f5f3	2026-11-04	**A0**	feat(dynamic-ux): add notification event foundation
02	5aed01e	2026-11-04	**A0**	fix(dynamic-ux): grant notification reads and refresh activity enrichment
03	45cf7e7	2026-11-04	**A1**	feat(dynamic-ux): add in-app banners, match banner, and message events
04	be90c0d	2026-11-04	**A1**	test(dynamic-ux): cover banner at 360 and 200% text
05	de4046e	2026-11-04	**A1**	fix(dynamic-ux): wire banner actions and isolate live state
06	c480fa7	2026-11-04	**A1**	fix(dynamic-ux): recover realtime channels after failed joins
07	06f10d8	2026-11-04	**A1**	fix(dynamic-ux): clear joined flags before channel teardown
08	b96c571	2026-11-04	**A2**	feat(dynamic-ux): add named haptics and shared sound service
09	4de648a	2026-11-04	**A2**	fix(dynamic-ux): honor vibration preference and serialize sound init
10	75b1ff8	2026-11-04	**A3**	feat(dynamic-ux): add per-category notification preferences
11	72fe744	2026-11-04	**A3**	fix(dynamic-ux): make notification preferences honest and consistent
12	f89f540	2026-11-04	**A3**	fix(dynamic-ux): scope quiet hours to notification feedback
13	dc5c03c	2026-11-05	**A4**	feat(dynamic-ux): add local notifications and permission flow
14	a2c3fc6	2026-11-05	**A4**	fix(dynamic-ux): harden local notifications and iOS delivery
15	ed9c0ed	2026-11-05	**A4**	fix(dynamic-ux): honor sound and vibration per notification
16	8d8f9e6	2026-11-05	**A6**	fix(dynamic-ux): unify feedback and remove raw error copy
17	445c4c9	2026-11-05	**A6**	fix(dynamic-ux): adopt feedback helper across changed flows
18	5b97786	2026-11-05	**A6**	fix(dynamic-ux): block technical error leakage and cover digit validation
19	da7ff16	2026-11-05	**A6**	test(dynamic-ux): cover technical error rejection and digit validation
20	cb7b96a	2026-11-05	**B0**	feat(qol): add refresh, offline state, badges, and drafts
21	27c3355	2026-11-05	**B0**	fix(qol): keep the offline strip out of the app bar
22	9df1965	2026-11-05	**B0**	fix(qol): preserve lists and callbacks during refresh
23	a24048f	2026-11-05	**B1**	feat(chat): add per-chat mute and manual unread
24	4b18f8a	2026-11-05	**B1**	feat(chat): add unread divider, scroll FAB, copy, search, and retry
25	fdd641b	2026-11-05	**B1**	fix(chat): own the search controller and wrap menu labels
26	a25dacd	2026-11-05	**B2**	feat(qol): add chat actions and activity controls
27	b8419ca	2026-11-05	**B2**	fix(qol): surface failures for clear-read and chat state

RANGE

3ab6700..08de0e8

TOTAL

54 commits · 94 files

DOCS EXCLUDED

Pages after 08de0e8 are documentation only.

COMMITS BY TRACK

TRACK	COMMITS	PHASES
A	19	A0–A6 · foundation, banners, feedback, local delivery
B	16	B0–B3 · primitives, chat, lists, polish
C	18	C0–C3 · contracts, presence, typing, receipts
D	1	D-01 · what's new

READING THE LEDGER

Hashes are abbreviated, dates are author dates, and the phase column maps each commit to the work item that closed it. The range is frozen at 08de0e8; documentation commits are deliberately absent.

Counts reconcile: 19 Track A commits + 16 Track B + 18 Track C + 1 Track D = 54. A7 is verification-only and contributes no commit.

COMMIT LEDGER II

Commits 28–54 of 54.

3ab6700..08de0e8
Newest last.

#	HASH	DATE	PHASE	SUBJECT
28	af0f3c4	2026-11-05	B2	fix(activity): keep the title, targets, and unread swipe
29	2491fc3	2026-11-05	B3	feat(polish): add resume setup, profile nudge, and empty actions
30	8110b51	2026-11-05	B3	fix(polish): scope resume state and respect completion
31	3e11df1	2026-11-05	B3	test(polish): cover completion gate and scoped keys
32	12afa6e	2026-11-05	B3	fix(polish): keep nudges in layout and unify empty actions
33	26d38a0	2026-11-05	B3	fix(polish): wrap nudge actions and drop premature dismissal
34	708316e	2026-11-05	B3	fix(polish): show the profile nudge through the app banner
35	bb54994	2026-11-05	B3	fix(requests): open sent requests as a backable page
36	a79ed63	2026-11-05	C0	feat(privacy): add chat status contracts and settings
37	5c1a4fd	2026-11-05	C0	fix(privacy): mask status data for non-pals and receipts off
38	4aaaa02	2026-11-05	C0	fix(privacy): make read timestamps rpc-only
39	4021a43	2026-11-05	C0	fix(privacy): block editing until state loads and reword promise
40	3fa4a45	2026-11-05	C0	fix(privacy): coalesce in-flight privacy loads
41	69bec41	2026-11-05	C1	feat(presence): add heartbeat, last-seen, and live online state
42	c8bb845	2026-11-05	C1	fix(presence): keep heartbeat fresh and gate the channel on loaded privacy
43	38949d7	2026-11-05	C1	fix(presence): decouple refresh, protect contrast, and scope online
44	b876970	2026-11-05	C1	fix(presence): stack row status and recover dropped channels
45	487b85a	2026-11-05	C1	fix(presence): fully tear down failed channels
46	5829b91	2026-11-05	C1	fix(presence): stop re-entrant channel teardown
47	27f4784	2026-11-05	C1	fix(presence): guard refresh after sign-out
48	354ff68	2026-11-05	C2	feat(typing): broadcast throttled typing on the chat channel
49	8da9b57	2026-11-05	C2	fix(typing): recover typing-only channels and show in empty chats
50	5bb26dc	2026-11-05	C2	fix(typing): keep the empty match card while a peer types
51	879e593	2026-11-05	C3	feat(receipts): broadcast and show seen state
52	3f468e5	2026-11-05	C3	fix(receipts): use the tested payload helper and pause guard
53	98e3b59	2026-11-05	C3	fix(receipts): trailing read flush and server-authoritative seen
54	08de0e8	2026-11-05	D	feat(polish): add one-time what's new sheet

RANGE

3ab6700..08de0e8

TOTAL

54 commits · 94 files

DOCS EXCLUDED

Pages after 08de0e8 are documentation only.

LARGEST PHASE

B3 · seven commits

Contextual polish carried the most commit-level iteration because it touched six screens and three empty-state families.

TEST GROWTH

80 → 155

Seventy-five tests were added across ten areas; no test was skipped and the suite was green at every phase close.

SCOPE BOUNDARY

Source only

The HTML, PDF and generator commits that follow this range are documentation and are not counted as product work.

Traceability: every phase page in this record names its commits; every commit in this ledger maps back to a phase. Nothing in the range is unexplained.

FILE INVENTORY I

Paths 1–47 of 94.

A/M/D as recorded by the source diff.

STATUS	PATH	AREA
M	android/app/build.gradle.kts	android
M	android/app/src/main/AndroidManifest.xml	android
A	android/app/src/main/res/drawable/ic_stat_palturo.xml	android
M	ios/Flutter/AppFrameworkInfo.plist	ios
M	ios/Runner.xcodeproj/project.pbxproj	ios
M	ios/Runner/AppDelegate.swift	ios
M	lib/authentication/auth_gate.dart	lib
M	lib/main.dart	lib
A	lib/models/app_notification.dart	lib
A	lib/models/incoming_message_event.dart	lib
M	lib/onboarding/onboarding_complete.dart	lib
M	lib/onboarding/onboarding_flow.dart	lib
M	lib/screen/account/account_settings_page.dart	lib
M	lib/screen/account/change_password_page.dart	lib
M	lib/screen/account/delete_account_page.dart	lib
M	lib/screen/action_dialogs.dart	lib
M	lib/screen/activity_presentation.dart	lib
M	lib/screen/block_list.dart	lib
M	lib/screen/card/request_card.dart	lib
M	lib/screen/chats_page.dart	lib
M	lib/screen/chats_room.dart	lib
M	lib/screen/explore_expand.dart	lib
M	lib/screen/explore_page.dart	lib
M	lib/screen/home_page.dart	lib
M	lib/screen/matches.dart	lib
M	lib/screen/navigation_bar.dart	lib
M	lib/screen/notifications_page.dart	lib
M	lib/screen/profile_edit.dart	lib
M	lib/screen/profile_page.dart	lib
D	lib/screen/request_match_sheet.dart	lib
M	lib/screen/request_page.dart	lib
M	lib/screen/settings.dart	lib
A	lib/screen/settings/chat_privacy_page.dart	lib
A	lib/screen/settings/notification_preferences_page.dart	lib
M	lib/screen/users_dump.dart	lib
M	lib/screen/utils/chat_unread.dart	lib
M	lib/screen/utils/filter_bottom_sheet.dart	lib
D	lib/services/activity_service.dart	lib
A	lib/services/app_badge_service.dart	lib
A	lib/services/app_navigation.dart	lib
A	lib/services/app_navigator.dart	lib
A	lib/services/app_shell_state.dart	lib
A	lib/services/chat_privacy_service.dart	lib
M	lib/services/chat_state_service.dart	lib
A	lib/services/connectivity_service.dart	lib
A	lib/services/draft_service.dart	lib
M	lib/services/haptics.dart	lib

SOURCE OF TRUTH

`git diff --name-status 3ab6700..08de0e8`

FILES

94 changed · 9,635 insertions · 1,301 deletions

SCOPE

Documentation commits after the source freeze are excluded.

FILE INVENTORY II

Paths 48–94 of 94.

A/M/D as recorded by the source diff.

STATUS	PATH	AREA
A	lib/services/local_notification_service.dart	lib
M	lib/services/match_api.dart	lib
A	lib/services/notification_presenter.dart	lib
A	lib/services/notification_service.dart	lib
M	lib/services/preferences_service.dart	lib
A	lib/services/sound_service.dart	lib
M	lib/utils/friendly_error.dart	lib
A	lib/utils/presence_label.dart	lib
A	lib/widgets/app_banner.dart	lib
A	lib/widgets/app_feedback.dart	lib
A	lib/widgets/chat_menu_row.dart	lib
A	lib/widgets/chat_mute_sheet.dart	lib
A	lib/widgets/chat_search_sheet.dart	lib
A	lib/widgets/match_banner.dart	lib
A	lib/widgets/offline_banner.dart	lib
A	lib/widgets/resume_setup_card.dart	lib
A	lib/widgets/typing_indicator.dart	lib
A	lib/widgets/whats_new_sheet.dart	lib
M	pubspec.lock	root
M	pubspec.yaml	root
A	supabase/migrations/20261105000100_notifications_realtime.sql	supabase
A	supabase/migrations/20261105000200_notification_context.sql	supabase
A	supabase/migrations/20261105000300_connection_preferences.sql	supabase
A	supabase/migrations/20261105000400_clear_read_notifications.sql	supabase
A	supabase/migrations/20261105000500_chat_privacy.sql	supabase
A	supabase/migrations/20261105000600_privacy_status_masking.sql	supabase
A	supabase/migrations/20261105000700_lock_connection_reads.sql	supabase
M	test/activity_presentation_test.dart	test
A	test/activity_qol_test.dart	test
A	test/app_banner_test.dart	test
A	test/app_feedback_test.dart	test
A	test/app_notification_test.dart	test
A	test/b3_polish_test.dart	test
A	test/chat_privacy_test.dart	test
A	test/chat_qol_test.dart	test
A	test/chat_widgets_test.dart	test
A	test/draft_service_test.dart	test
M	test/friendly_error_test.dart	test
A	test/haptics_test.dart	test
A	test/incoming_message_event_test.dart	test
A	test/local_notification_payload_test.dart	test
A	test/notification_preferences_test.dart	test
A	test/offline_banner_test.dart	test
A	test/qol_services_test.dart	test
D	test/request_match_sheet_test.dart	test
A	test/sound_service_test.dart	test
A	test/whats_new_test.dart	test

SOURCE OF TRUTH

`git diff --name-status 3ab6700..08de0e8`

FILES

94 changed · 9,635 insertions · 1,301 deletions

SCOPE

Documentation commits after the source freeze are excluded.

LIMITATIONS

What is explicitly not claimed.

Documented, owned, outside the shipped scope.

The release has one external blocker and five accepted boundaries. None is an unresolved defect inside the sixteen verified phases, but each changes what the product may safely promise.

ITEM	STATUS AND HANDLING
A5 push	Blocked on Firebase project/config, APNs key, web VAPID, and a service account. Killed-app alerts and guaranteed iOS background delivery are unavailable. In-app, Activity, badges, foreground banners, local notifications while alive, and server reconciliation remain.
Realtime topic ACL	The management role cannot alter realtime.messages (owned by supabase_realtime_admin). chat:<connectionId> topics rely on unguessable UUIDs and client-side privacy gating. Add a dashboard-authored policy if server-side ACLs are required.
Read-receipt transport	C3 uses a read broadcast + 60s conversation refresh instead of publishing connections, because C0 made connection read timestamps RPC-only to honor the receipt toggle.
iOS build	Source/config verified but not compiled on this Windows host. A macOS build and device pass are still required.
D-02 review prompt	Deferred until a store listing exists.
Background receipts	A read event can arrive before mark_messages_read commits; server refresh reconciles within the next poll.

Release hold: none inside the shipped scope.

A5 is a delivery gap, not a hidden failure. The correct public claim is in-app notifications, Activity, badges and local alerts while the process is alive — not killed-app push.

OPERATIONAL FOLLOW-UPS

- Add Firebase, APNs, VAPID and a service account to unblock A5.
- Compile and device-test iOS on macOS.
- Add a dashboard realtime topic policy if server-side ACLs are required.

PRODUCT FOLLOW-UPS

- D-02 waits for a real store listing.
- Background receipts reconcile within the next poll.
- Consider a broader read for presence if product wants roots, not leaves.

SAFE RELEASE WORDING

CLAIM	STATE	USE INSTEAD
Killed-app push	NOT SHIPPED	“Notifications work in-app and while the Android process is alive.”
Guaranteed iOS background	NOT SHIPPED	“Local alerts are wired; device delivery still needs a macOS build.”
Server-side realtime ACL	PARTIAL	“Topics use unguessable connection ids and client privacy gating.”
Store review prompt	DEFERRED	“Planned once a published store listing exists.”

A5 unblock checklist

Firebase project · Android/iOS/web apps · APNs .p8 · VAPID key · service account. No client redesign is required.

Device QA

A macOS build and a physical VoiceOver/TalkBack pass remain recommended before public release.

What is not on this list: crashes, data loss, unrecoverable states or silent write failures inside the shipped surfaces. The limitations are scope boundaries and external dependencies, not hidden defects.

OPERATIONS

What the credentials unlock.

A5 is an infrastructure task, not a redesign.

Nothing in the client needs to be re-architected for push. The remaining work is identity: a Firebase project, platform configuration files, an APNs key, a web VAPID key and a service account for the sender.

REQUIRED INPUTS

CREDENTIAL	PURPOSE
`Firebase project`	Android, iOS, web apps registered to the release application id.
`android/app/google-services.json`	Android Firebase identity.
`ios/Runner/GoogleService-Info.plist`	iOS Firebase identity.
`APNs .p8 + Key ID + Team ID`	Apple push authorization.
`Web VAPID public key`	Browser push subscription identity.
`Firebase service account JSON`	Edge function authentication to FCM HTTP v1.

SAFE STATE TODAY

- ✓ In-app banners route correctly
- ✓ Activity receives realtime inserts
- ✓ Local alerts work while alive
- ✓ Badges reflect unread totals
- ✓ Resume reconciles missed events
- ✓ No killed-app claim is made

WHEN CREDENTIALS ARRIVE

01
Register the apps

Create the Firebase project and register the release application id for Android, iOS and web.

02
Place identity files

Add `google-services.json`, `GoogleService-Info.plist`, APNs .p8 and the VAPID key.

03
Deploy the sender

Add the service account to the edge function and send through FCM HTTP v1 with the same event kinds.

Order matters: the topic, payload and routing contract already exist. A5 should reuse them; it should not invent a second notification model.

ROLLOUT SEQUENCE

01 Configure
Firebase project and the three app registrations.

02 Install identity
Place the Android, iOS and web credential files.

03 Send
Deploy the FCM HTTP v1 edge function and token refresh.

04 Verify
Test killed-app delivery on Android and iOS before claiming it.

POST-ROLLOUT VERIFICATION

CHECK	PASS CONDITION
Token registration	Token stored, refreshed, and removed on sign-out
Foreground	One alert only; in-app banner remains primary
Background	Local alert follows the same category and quiet-hours gates
Killed app	Push received and routed to the correct surface

CLIENT CHANGES

Small: token registration and foreground handler.

SERVER CHANGES

One sender path in an edge function.

RELEASE CLAIM

Update only after device verification.

AUDIT LEDGER

Every phase has three independent sessions.

Functionality, quality, regression. No phase advanced until all three agreed.

Audits were read-only. Each reviewer inspected the implemented tree, the tests and the live state without editing files. Session IDs are recorded verbatim below so the review trail can be reopened.

PHASE	DATE	FUNCTIONALITY	QUALITY	REGRESSION	COMMIT
A0	2026-11-04	ses_e592a7f30ffeq8fUwgk2ANDfUM	ses_e592a7f2effeLn8rVWs2rrQPbq	ses_e592a7f2cffe8GfwwpD640Klre	5aed01e
A1	2026-11-04	ses_e5901577affeq2wa4Giny1Zc2G	ses_e59015779ffeOb8RHyB6BUeNZe	ses_e59015777ffeEdT9T7s7tkrkxz	06f10d8
A2	2026-11-04	ses_e58dab2d7ffeux946Z2eIEhyzD	ses_e58dab2d5ffezFyNm7D75VvxET	ses_e58dab2d3ffeV5XsamgW1U0F5L	4de648a
A3	2026-11-05	ses_e58c5095fffegIgQeovEXrHzdE	ses_e58c5095effeUezhvpt8d7r2sM	ses_e58c5095cffegDv70umgaruv8T	f89f540
A4	2026-11-05	ses_e58adc0d1ffeVWtFpai92ukhG8	ses_e58adc0ceffedzJvP1urIo5eUn	ses_e58adc0caffe3Ux8H0s31HmBs0	ed9c0ed
A6	2026-11-05	ses_e588dbc5affeGDZTXb8Pmwvtnw	ses_e588dbc48ffeXisBr5i8JIysPv	ses_e588dbc45ffesfpI7VGMo097lk	5b97786
B0	2026-11-05	ses_e5843f5e2ffefz7EXGxqIi7bLv	ses_e5836b58bffeKweLZoQIitxgHT	ses_e58300b0bffeTYwA6tCsWyiY2M	9df1965
B1	2026-11-05	ses_e58162853ffeY6eXrwW9epCzOH	ses_e58102699ffe7qOQlEUSgH3ZZD	ses_e58000898ffe0vOEXUSh9pzpQy	fdd641b
B2	2026-11-05	ses_e57ef735fffeE9Nt6jlx4U5ATu	ses_e57e2ef40ffeWacaCAK9NOF8US	ses_e57d3680bffeb5fsW9dx9hHZ45	af0f3c4
B3	2026-11-05	ses_e57c785d2ffe2ZSlK7cqlhTQff	ses_e57bc1a33ffefghRV09XmL0825	ses_e57920c5bffeSS8Rni62EB084P	bb54994
C0	2026-11-05	ses_e577e7e0bffeGsh00O8TG4Aap5	ses_e5771c409ffeX81Tj3KRgUxjeZ	ses_e575d14d5ffeer0oF029QXVh3t	3fa4a45
C1	2026-11-05	ses_e56cb7616ffekYx0WUloT2M0zV	ses_e56bd7611ffeNnX2Oe0ucXYyKl	ses_e56a791a8ffesn8nk0lS0zWOFT	5829b91
C2	2026-11-05	ses_e5699eeebffef3BcJw4yGqjhig	ses_e56930aa7ffe3BEpK6pz3eqfd4	ses_e568a4196ffeROc75CIa5nBffl	5bb26dc
C3	2026-11-05	ses_e567e3b64ffe16SQgNbF6cUg0W	ses_e56783fa9ffeX0OJ37fR5RdQMt	ses_e5669814 7ffezN0ys13sCwpNYN	98e3b59

42
AUDIT SESSIONS

14
PHASES REVIEWED

0
BLOCKING FINDINGS

Coverage note: D-01 closed on the full suite and a live once-per-version probe. A7 was verification-only and has no separate audit triplet; its gates are recorded in the release section.

WHAT AUDITORS CHECKED

- Acceptance intent, primary and failure paths
- Real server contracts and live probes
- Dark mode, 200% text, 360/390/1440
- Adjacent flows and regression suite

WHAT CLOSED A FINDING

- A reproducing test or live probe
- A fix in the same phase
- Re-run of the full suite
- A second reviewer pass

AUDIT OUTCOMES

RESULT	MEANING	COUNT
Pass	All three lenses agreed; phase closed	14 phases
Fix + re-audit	Blocking finding fixed before close	Recorded in phase pages
Verification only	A7 gates, no separate audit triplet	1 phase
Blocked	A5 credentials absent; not reviewed as shipped	1 item

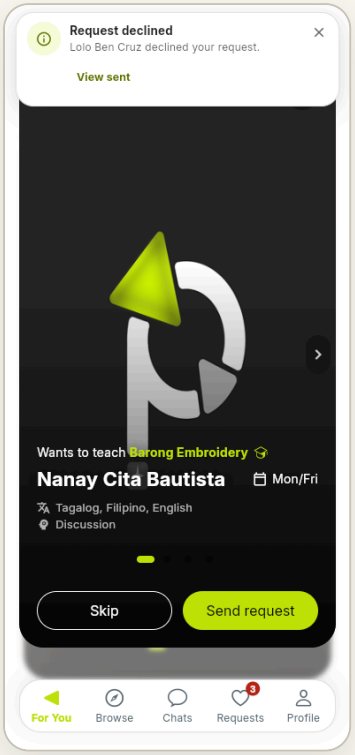
Independence: the three reviews were read-only and did not edit the tree. A phase only advanced when functionality, quality and regression all passed.

VISUAL EVIDENCE 01 / 05

Live events and notification controls.

Captured from release and debug builds.
390 × 844 unless noted.

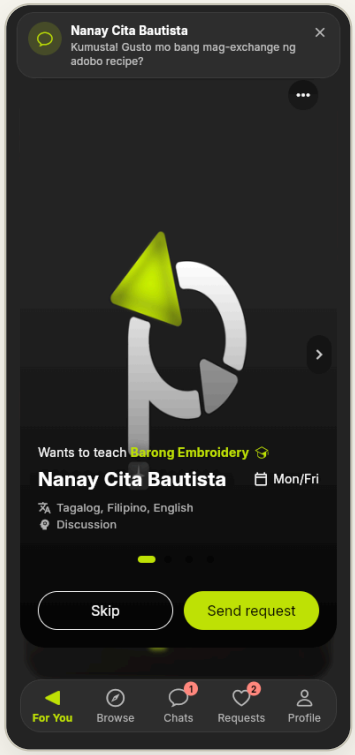
Three banner states and the notification preferences page in both themes. Banners sit at the top, preserve the current screen, and expose one route-aware action.



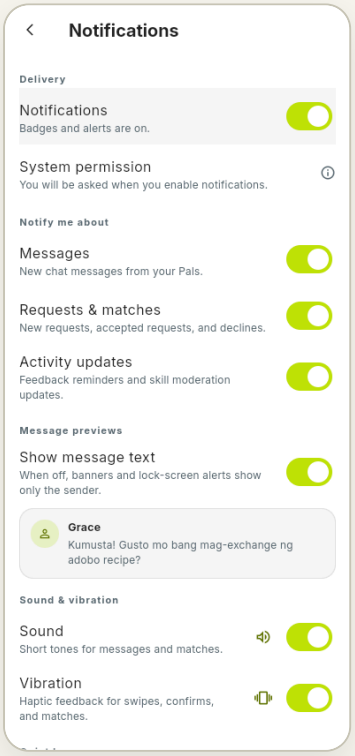
Request declined
Top banner with a route back to Requests.



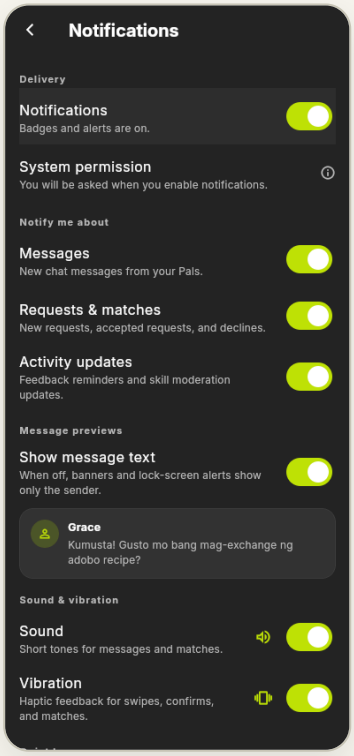
New match, dark
Skill line and Say hello; once per connection.



New message
Sender, preview and tap-to-chat.



Preferences, light
Delivery, categories, preview, sound, quiet hours.



Preferences, dark
The same hierarchy in the charcoal theme.

What to look for

- Banner clears the status bar and app bar.
- Exactly one primary action per banner.
- Dark mode keeps the lime accent and contrast.
- Settings reports real permission state, not a promise.

Frames are ordered as captured. No state was reconstructed for this document.

FRAME INDEX

01	Declined banner
02	Match, dark
03	Message, dark
04	Preferences, light
05	Preferences, dark

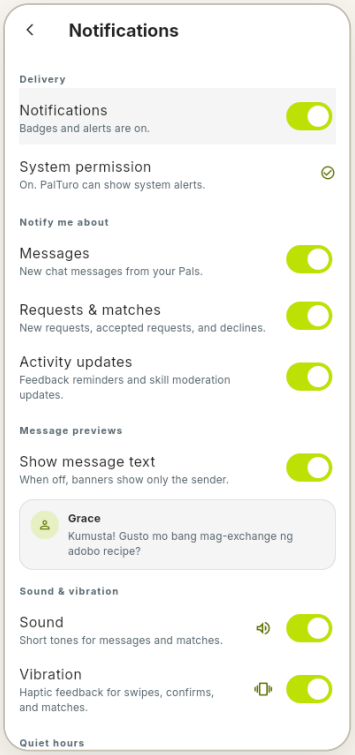
Capture set · 5 frames
390 × 844 · release web
5 November 2026

VISUAL EVIDENCE 02 / 05

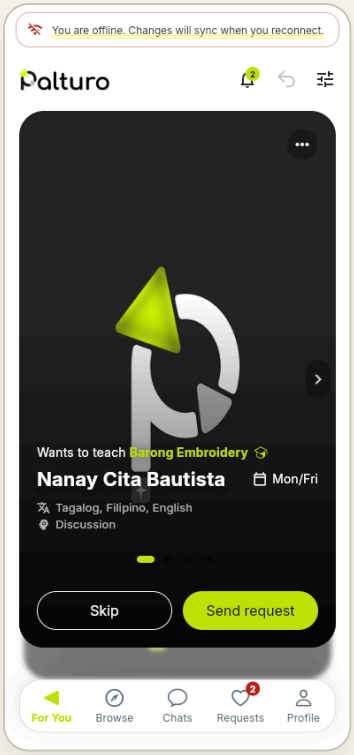
Permissions, connectivity and drafts.

States that are easy to fake.
These are captured from real transitions.

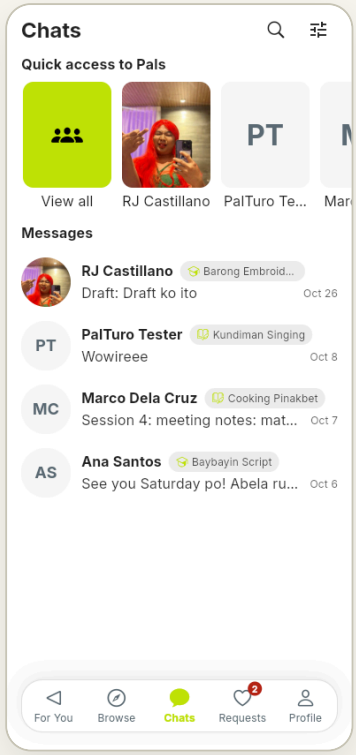
The permission surface distinguishes not-asked from denied; the offline strip occupies layout space above the app bar; drafts survive leaving the composer; the chat menu exposes the full action set.



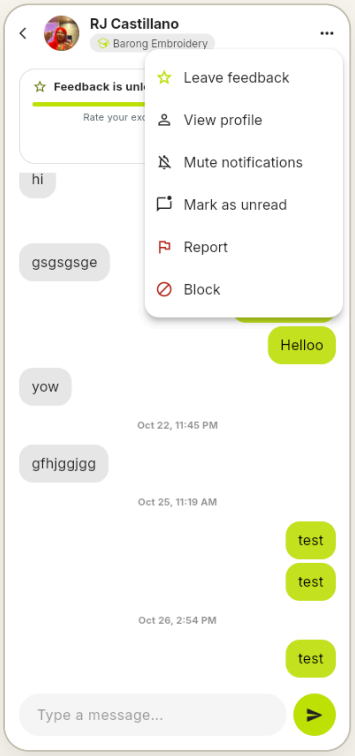
Permission state
Real system state with the correct next action.



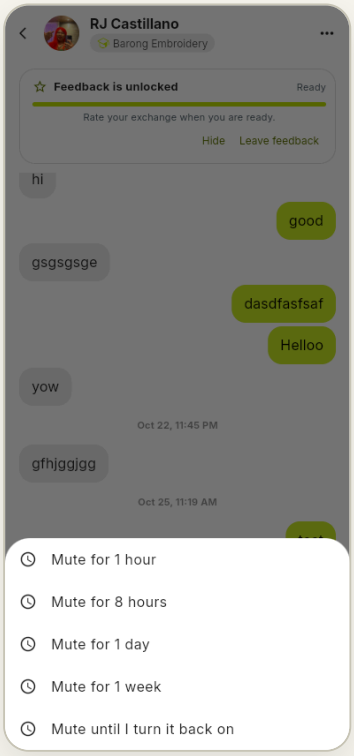
Offline strip
Above the app bar; navigation stays tappable.



Draft preview
Per-chat draft after leaving the composer.



Chat menu
Profile, feedback, mute, unread, search, report, block.



Mute durations
One hour through until turned back on.

State honesty

- Permission copy changes with the OS answer.
- Offline never covers a control.
- Draft text is scoped to one conversation.
- Mute is server-persisted, not a local flag.

Each frame is a live transition, not a mocked screen. Demo profiles are fictional.

FRAME INDEX

01	Permission state
02	Offline strip
03	Draft preview
04	Chat menu
05	Mute durations

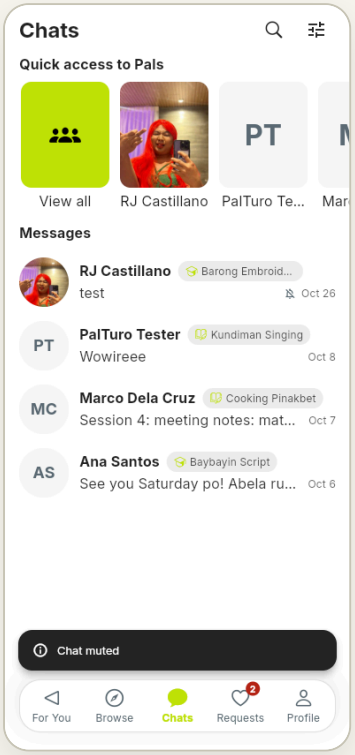
Capture set · 5 frames
390 × 844 · Android debug
5 November 2026

VISUAL EVIDENCE 03 / 05

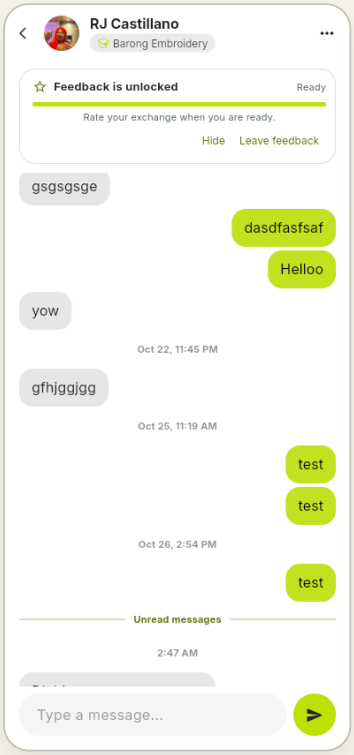
Chat controls in use.

Mute, unread, navigation, search, copy.
All from the same room.

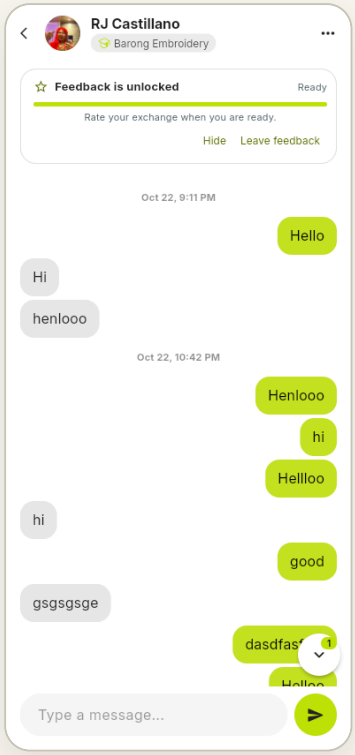
The room now carries the management affordances that make a long thread navigable: a muted-state list, a first-unread divider, a counted scroll FAB, in-room search and a single copy confirmation.



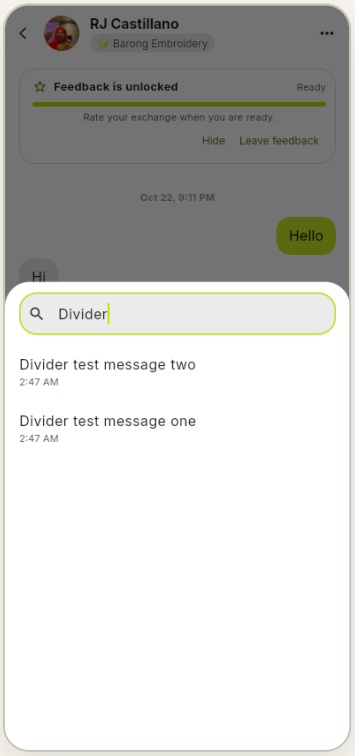
Muted chat
Mute icon retained; unread still counts.



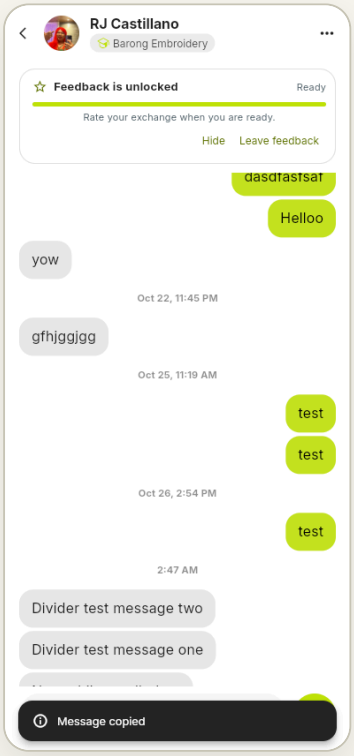
Unread divider
The first unread message gets a boundary.



Scroll FAB
New-message count while scrolled up.



In-chat search
Results over the loaded message window.



Copy confirmation
One snackbar after the clipboard write.

Room rules

- Auto-scroll only when already at the bottom.
- Muted chats suppress alerts, not unread.
- Search states its scope.
- Copy does not double-toast.

The room is shown at 390 with the real composer and navigation present.

FRAME INDEX

01	Muted list
02	Unread divider
03	Scroll FAB
04	In-chat search
05	Copy confirmation

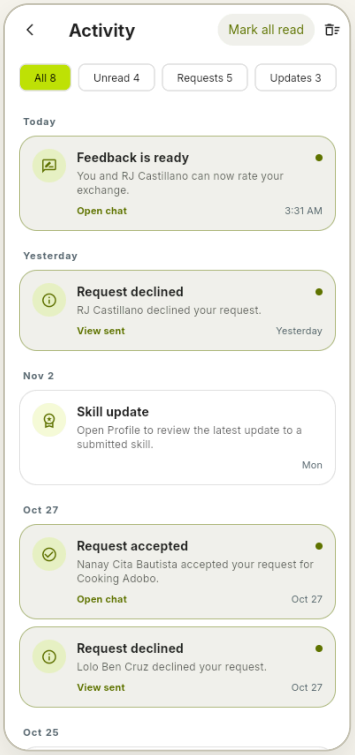
Capture set · 5 frames
390 × 844 · release web
5 November 2026

VISUAL EVIDENCE 04 / 05

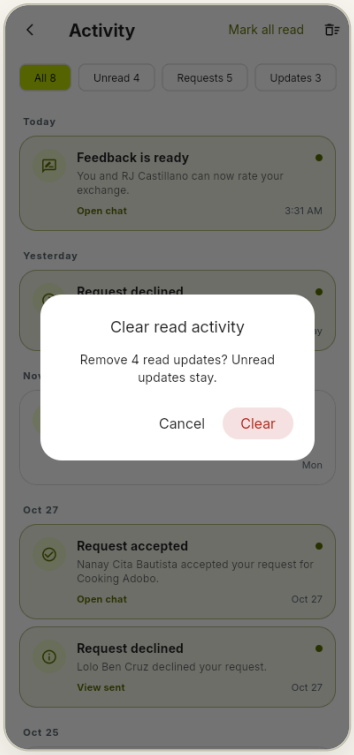
Lists, Activity and contextual polish.

Triage from the row.
One prompt at a time.

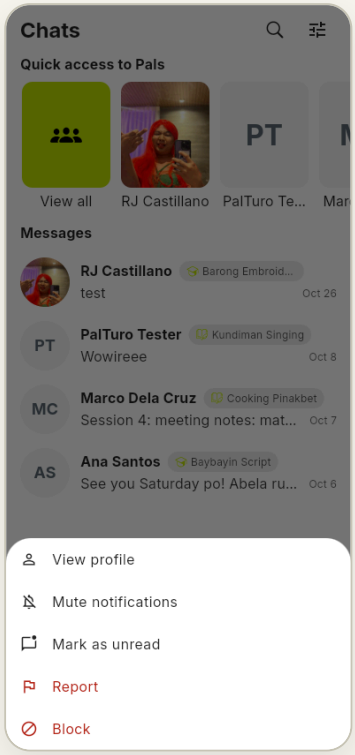
Activity filters and clear-read act on real rows; the chat row exposes the same actions as the room; the profile nudge and the version-gated what's-new sheet stay non-blocking.



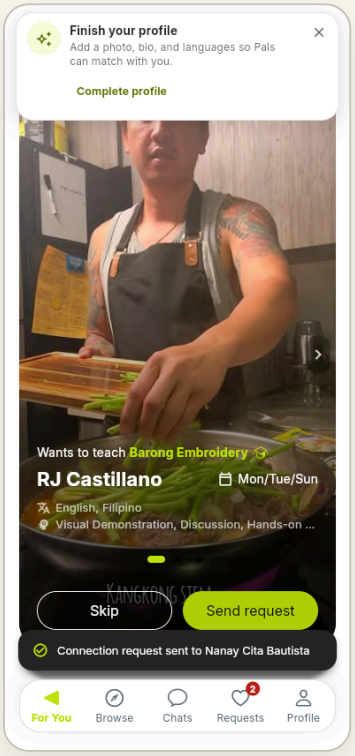
Activity filters
All, unread, requests and updates with counts.



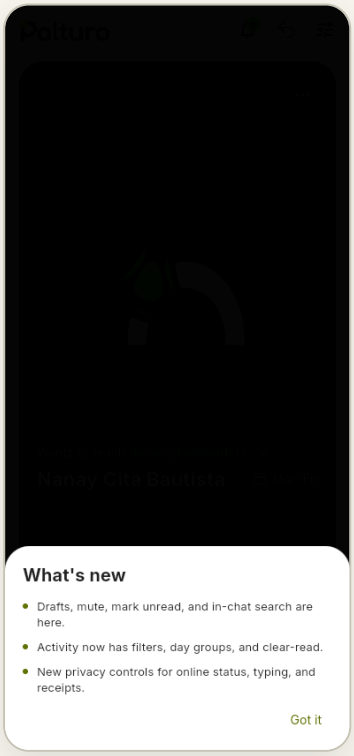
Clear read
Confirmation; unread rows are never deleted.



Chat long-press
Profile, mute, unread, report and block.



Profile nudge
After a committed request; once per session.



What's new
One sheet per version, dismiss persists.

Prompt discipline

- Never block a completed action.
- Never stack two completion prompts.
- Every empty state has one action.
- Dismissal is scoped per user or version.

List and prompt states are shown together to make the interruption budget visible.

FRAME INDEX

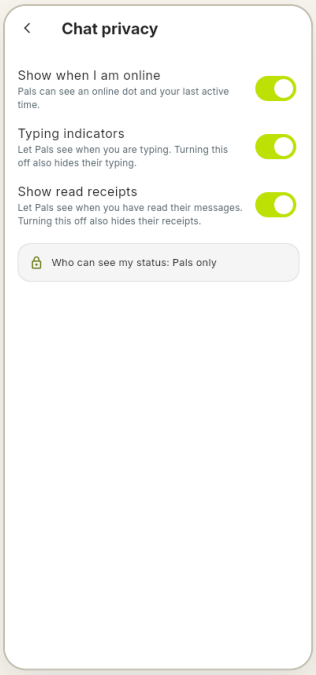
01	Activity filters
02	Clear read
03	Chat long-press
04	Profile nudge
05	What's new

Capture set · 5 frames
390 × 844 · release web
5 November 2026

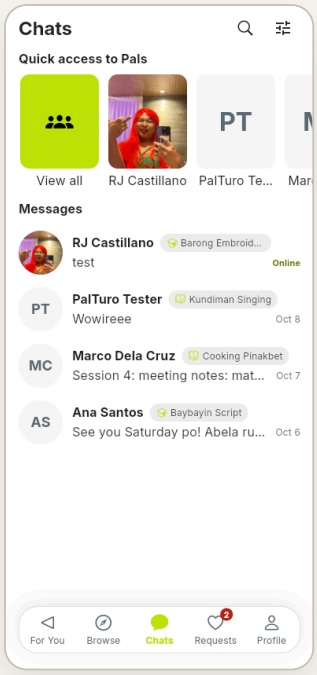
VISUAL EVIDENCE 05 / 05

Privacy, presence and read receipts.

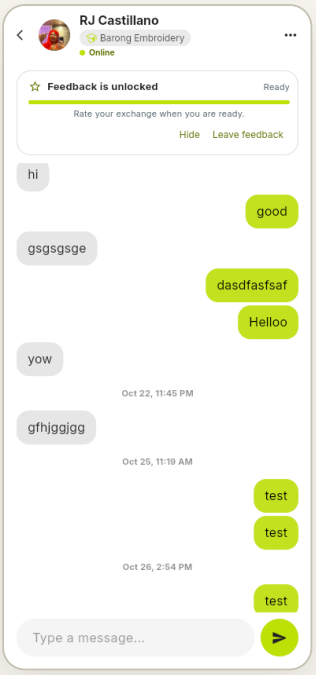
The status layer, as a peer sees it.
Masked at the server.



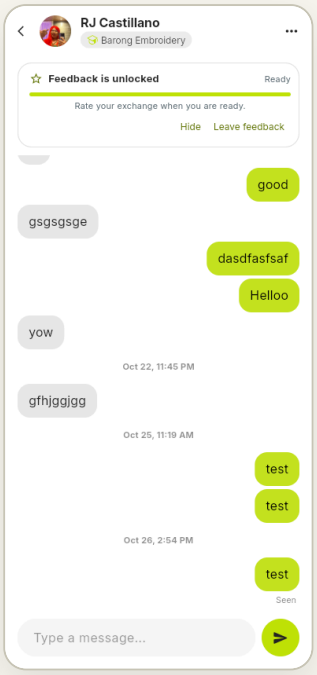
Chat privacy
Online, typing, receipts, Pals only.



Online in the list
Message time remains visible.



Online in the room
Under the peer name.



Seen
Only the last own message, server time.

PRESENCE STATE MACHINE

Allowed	Track; heartbeat every 60s; show Online or last active.
Off	No track, no heartbeat, no status returned.
Not loaded	Controls disabled; no optimistic track.
Leave / pause	Remove channel; stop heartbeat.
Error	Teardown once; retry without re-entrancy.

RECEIPT RULES

- ✓ Seen only under the last own message.
- ✓ Only when the peer allows receipts.
- ✓ Only when the server timestamp covers it.
- ✓ Broadcast is a hint; server reload is the source.
- ✓ Backgrounded broadcasts pause; 60s refresh reconciles.

Consistency: the privacy page, the list and the room all read the same server-masked contract. Turning presence off removes the status everywhere, not just where the toggle was rendered.

VISIBILITY BY SETTING

SETTING	LIST	ROOM	RECEIPTS
All on	Online / Active	Online	Seen
Presence off	Message time	No status	Seen if allowed
Typing off	—	No dots	Seen if allowed
Receipts off	—	No Seen	Hidden

Peer view

The list and room never show a status the server did not return for that viewer.

Local view

The owner always sees their own switch state, even when the peer sees nothing.

One contract, three surfaces

The privacy page writes the flags, the conversation RPC masks them per viewer, and the list and room render only what the RPC returned. There is no local “pretend offline” path.

ACCEPTANCE

Sixteen verified phases. One honest gap.

Final position at 08de0e8.

16 / 16

verified phases accepted within the shipped scope

Every planned item except A5 and D-02 has a working implementation, a quality review, a regression review, a commit and recorded verification. The release build is current and the tree is clean.

BRANCH	**ui-features @ 08de0e8**
PHASES	**16 verified · 1 blocked · 1 deferred**
AUDITS	**42 sessions · 0 blocking findings**
TESTS	**155 / 155**
ANALYZER	**clean**
MIGRATIONS	**7 applied live**
COMMIT RANGE	**3ab6700..08de0e8 · 54 commits**
PUSH DELIVERY	**Blocked on credentials**

RELEASE STATEMENT

Notifications work in-app and through a background process on Android. Activity, badges, banners, local alerts and server reconciliation are verified. Killed-app delivery and guaranteed iOS background delivery are not claimed.

D-02 remains deferred until a store listing exists. **A5** remains blocked until Firebase, APNs, VAPID and a service account are supplied.

The product is quieter, more legible, and more truthful about what it knows.

Acceptance is based on three independent audits per phase, live database probes, the final suite, both builds, and the captures in this record.

RELEASE READINESS

AREA	STATE	EVIDENCE
In-app notifications	READY	Banner probes, widget tests
Activity and badge	READY	Filters, clear-read, badge math
Chat quality of life	READY	Mute, drafts, divider, search
Privacy and presence	READY	ACL probes, masking tests
Killed-app push	BLOCKED	A5 credentials

Ships

Banners, Activity, local alerts while alive, badges, drafts, mute, unread, search, presence, typing, receipts and what's new.

Does not ship

Killed-app push, guaranteed iOS background delivery and the store review prompt. Each is recorded with its dependency.

PRODUCT / DESIGN ACCEPTANCE

ENGINEERING VERIFICATION · 5 NOVEMBER 2026

Palturo

END OF REPORT

One event system.
Sixteen verified phases.

Dynamic UX, QOL & social presence · implementation & verification

ui-features @ 08de0e8 · 5 November 2026